

Chapter 69: Signals

Scenario 1: Refactoring a BehaviorSubject-based cart service to signals

Situation: Your team has a `CartService` built entirely on RxJS: a `BehaviorSubject<CartItem[]>` holds state, an `asObservable()` is exposed to components, and every component subscribes in the template with `async` pipe. Management wants to move toward zoneless change detection and asks you to modernize this service using signals.

Question: How would you refactor this service to use signals, and what subtle behavioral differences must you watch out for compared to the RxJS version?

Answer: Replace the `BehaviorSubject` with a `writableSignal`, expose a read-only `Signal` via `.asReadonly()`, and derive totals with `computed()` instead of `combineLatest + map`. The key behavioral differences: signals are synchronous and glitch-free (no need for `distinctUntilChanged`, but you DO need to be careful about mutating arrays in place — signals compare by reference, not by deep equality, so `items.push(x)` followed by `set(items)` won't notify unless you create a new array). Also, signals have no concept of "completion" or "error channel" — error states must be modeled explicitly as part of the state shape.

```
interface CartItem { id: string; qty: number; price: number; }

@Injectable({ providedIn: 'root' })
export class CartService {
  // Private mutable state
  private readonly _items = signal<CartItem[]>([]);

  // Public read-only view – consumers cannot call .set()/.update()
  readonly items = this._items.asReadonly();

  // Derived state – recomputed only when _items actually changes reference
  readonly itemCount = computed(() =>
    this._items().reduce((sum, i) => sum + i.qty, 0)
  );

  readonly total = computed(() =>
    this._items().reduce((sum, i) => sum + i.qty * i.price, 0)
  );

  addItem(item: CartItem): void {
    this._items.update(items => {
      const existing = items.find(i => i.id === item.id);
      if (existing) {
        return items.map(i =>
          i.id === item.id ? { ...i, qty: i.qty + item.qty } : i
        );
      }
      return [...items, item]; // new array reference – triggers update
    });
  }
}
```

```
removeItem(id: string): void {
  this._items.update(items => items.filter(i => i.id !== id));
}
}
```

The critical migration mistake teams make is keeping an `effect()` around to "sync" the signal back into a Subject for legacy consumers indefinitely — that's a crutch, not a migration. Instead, use `toObservable(cartService.items)` at the boundary only where a legacy RxJS API genuinely still needs a stream (e.g., feeding into `combineLatest` with an HTTP call), and plan to remove that adapter once the consumer is migrated too.

Interviewer intent: Tests whether the candidate understands reference-equality semantics of signals (a very common bug source) and whether they can distinguish a "real" migration from a "papered-over" one that keeps two state systems in permanent parallel.

Scenario 2: An effect() causing an infinite loop

Situation: A junior developer writes a component where an `effect()` reads a signal `count` and, based on some condition, calls `count.set(...)` to "correct" the value. In production, the app hangs and DevTools shows thousands of effect executions per second.

Question: Why does this cause an infinite loop, and what is the correct pattern to fix it?

Answer: `effect()` tracks every signal read synchronously during its execution. If the effect writes to a signal it also reads (directly or indirectly through a computed that depends on it), Angular re-runs the effect because the write invalidates the very dependency the effect just read — creating a cycle. Angular's dev-mode `effect()` will actually throw `NG0600: writing to signals is not allowed in a computed or effect...` in many cases, but if the write happens conditionally or via `untracked` misuse, it can still loop or silently corrupt state.

The fix is almost always: **don't use effect() to derive or correct state — use computed() instead**, or if a side effect on an external system (not another signal) is truly needed, use `untracked()` to read without creating a dependency, and make sure the write path can never re-trigger the same effect.

```
// ❌ WRONG - infinite loop / NG0600
export class BadCounterComponent {
  count = signal(0);

  constructor() {
    effect(() => {
      const c = this.count(); // read → dependency
      if (c > 10) {
        this.count.set(10); // write to the same signal → re-triggers effect
      }
    });
  }
}

// ✅ CORRECT - clamp via computed(), no side effect needed
export class GoodCounterComponent {
  private readonly _rawCount = signal(0);
  readonly count = computed(() => Math.min(this._rawCount(), 10));
}
```

```

increment(): void {
  this._rawCount.update(c => c + 1);
}
}

// ✅ If you truly need an effect to sync to a NON-signal external system,
// read the OTHER signal, not the one you're about to write, and use untracked
// for reads that must not create a dependency edge.
export class LoggingComponent {
  count = signal(0);
  constructor() {
    effect(() => {
      const value = this.count(); // tracked read, fine – we don't write `count` here
      untracked(() => this.analytics.log('count changed', value));
    });
  }
}

```

Interviewer intent: Checks whether the candidate truly understands effect's dependency-tracking model and the golden rule "effects are for side effects, computed is for values" — one of the most tested signal concepts in interviews.

Scenario 3: computed() vs effect() for a derived-value bug

Situation: A component shows a "shipping cost" that should always equal `baseShipping + (isExpress() ? expressSurcharge : 0)`. The original developer implemented this using an `effect()` that sets a separate `shippingCost` signal whenever `isExpress` or the base values change. QA reports that occasionally the displayed shipping cost is one render "behind" the express toggle, especially in a zoneless setup.

Question: Why does the effect-based derivation lag, and how should this be modeled correctly?

Answer: `effect()` runs asynchronously, scheduled as a microtask after signal writes occur (batched), not synchronously during the same read like `computed()`. So when `isExpress` flips, there is a small window/render pass before the effect fires and updates the `shippingCost` signal — in zoneless apps with `onPush` this can produce a visibly stale value or a double-render flicker. `computed()`, on the other hand, is a pure, lazily-evaluated, memoized derivation that is always consistent with its sources at the moment it's read — there is no lag because there's no separate "second signal" to keep in sync; the computed IS the derived value.

The rule of thumb: **if a value can be expressed as a pure function of other signals, it must be a `computed()`, never an `effect()` writing to a plain signal.** Reserve `effect()` strictly for actual side effects (DOM manipulation outside Angular's control, logging, localStorage sync, imperative third-party widget updates) — never for producing a value that the template renders.

```

// ❌ WRONG – effect-derived value, can lag and is unnecessarily indirect
export class ShippingBad {
  baseShipping = signal(5);
  isExpress = signal(false);
  expressSurcharge = signal(15);
  shippingCost = signal(5);

  constructor() {

```

```

    effect(() => {
      const cost = this.baseShipping() +
        (this.isExpress() ? this.expressSurcharge() : 0);
      this.shippingCost.set(cost); // async write, template can see stale value
    });
  }
}

// ✅ CORRECT – computed is synchronous, memoized, and always consistent
export class ShippingGood {
  baseShipping = signal(5);
  isExpress = signal(false);
  expressSurcharge = signal(15);

  shippingCost = computed(() =>
    this.baseShipping() + (this.isExpress() ? this.expressSurcharge() : 0)
  );
}

```

Interviewer intent: Directly probes the candidate's mental model of push-pull scheduling differences between `computed()` (pull, synchronous-on-read) and `effect()` (push, scheduled) — a frequent source of subtle production bugs.

Scenario 4: Signal-based reactive form validation pattern

Situation: You're building a signup form without Angular's `ReactiveFormsModule`, using plain signals for each field (a growing trend in signal-forms-style codebases) because the form is simple and the team wants to avoid `FormGroup` overhead. You need cross-field validation: "confirmPassword must equal password" and "username must be at least 4 chars and not already taken (async check)."

Question: Design a signal-based validation architecture for this form, including how you'd handle the async "username taken" check without causing race conditions.

Answer: Model each field as a signal, derive per-field and cross-field errors with `computed()`, and use the `resource()` API (or `rxResource`) for the async username-availability check so Angular manages cancellation/race conditions for you instead of hand-rolling `switchMap`.

```

@Component({ /* ... */ })
export class SignupFormComponent {
  username = signal('');
  password = signal('');
  confirmPassword = signal('');

  usernameError = computed(() => {
    const u = this.username();
    if (u.length === 0) return null;
    if (u.length < 4) return 'Username must be at least 4 characters';
    return null;
  });

  passwordMismatch = computed(() =>
    this.confirmPassword().length > 0 &&
    this.confirmPassword() !== this.password()
  );
}

```

```

);

// resource() re-runs its loader whenever `params` changes, and automatically
// cancels/ignores stale in-flight requests – no manual race-condition handling
usernameAvailability = resource({
  params: () => ({ username: this.username() }),
  loader: async ({ params, abortSignal }) => {
    if (params.username.length < 4) return null; // skip trivial calls
    const res = await fetch(
      `/api/check-username?u=${params.username}`,
      { signal: abortSignal }
    );
    return (await res.json()) as { available: boolean };
  },
});

isUsernameTaken = computed(() =>
  this.usernameAvailability.value()?.available === false
);

formValid = computed(() =>
  !this.usernameError() &&
  !this.passwordMismatch() &&
  !this.isUsernameTaken() &&
  this.usernameAvailability.status() !== 'loading'
);

submit(): void {
  if (!this.formValid()) return;
  // proceed with signup
}
}

```

Why not `effect()` for the async check? Because `effect()` is fire-and-forget for side effects — you'd need to manually track loading/error/cancellation state yourself, re-implementing exactly what `resource()` already does (AbortController wiring, request de-duplication on identical params, stale-response discarding). Using `resource()` here isn't just cleaner, it's more correct: a naive `effect()` that calls `fetch` on every keystroke and sets a signal on `.then()` is a classic race-condition bug (a slow early response can arrive after a fast later one and overwrite it with stale data).

Interviewer intent: Evaluates whether the candidate reaches for `resource()` for async-derived state instead of misusing `effect()`, and understands why race-condition-safety is a first-class reason to prefer the resource API over manual promise/effect wiring.

Scenario 5: Combining signals with async data via `toSignal` and `resource`

Situation: A `ProductDetailComponent` needs to fetch product data from an HTTP `observable<Product>` (from a legacy `ProductService.getProduct$(id)` that must remain RxJS-based due to shared interceptor logic), while the rest of the component is signal-based. The product ID comes from a route param signal.

Question: How do you bridge this legacy Observable-returning service into the signal graph cleanly, and what are the pitfalls of `toSignal()` you must account for?

Answer: Use `toSignal()` to convert the Observable into a signal, but critically the Observable must be re-created reactively whenever `productId` changes — usually via a `computed()` that returns an Observable is wrong (computed must be synchronous and pure); instead use `toObservable(productId)` piped through `switchMap`, or better, wrap the whole thing in `resource()` / `rxResource()` if targeting a version with resource support, since it natively accepts a reactive `params` function and handles cancellation.

```
@Component({ /* ... */ })
export class ProductDetailComponent {
  private route = inject(ActivatedRoute);
  private productService = inject(ProductService);

  // Route param as a signal
  productId = toSignal(
    this.route.paramMap.pipe(map(p => p.get('id')!)),
    { initialValue: null }
  );

  // Bridging legacy Observable-based service reactively
  private product$ = toObservable(this.productId).pipe(
    filter((id): id is string => id !== null),
    switchMap(id => this.productService.getProduct$(id))
  );

  product = toSignal(this.product$, { initialValue: null });

  // Preferred modern alternative if the service can expose a promise-friendly
  // or resource-compatible API:
  productResource = rxResource({
    params: () => ({ id: this.productId() }),
    stream: ({ params }) =>
      params.id ? this.productService.getProduct$(params.id) : of(null),
  });
}
```

Pitfalls of `toSignal()`:

1. **Initial value:** Until the first emission, the signal holds `undefined` unless you supply `initialValue`, which can cause template errors if you assume non-null.
2. **Injection context:** `toSignal()` must be called in an injection context (constructor, field initializer, or with an explicit `injector` option) — calling it inside a method later will throw.
3. **No automatic cancellation of prior requests** unless you build the `switchMap` chain yourself — this is exactly the case where `rxResource()` / `resource()` is superior, since it manages cancellation and stale-response discarding for you.
4. **Manual unsubscription is unnecessary** — `toSignal()` unsubscribes automatically when the injection context (component) is destroyed, but only if it was created in that context; if hoisted into a shared service incorrectly, it can leak.

Interviewer intent: Tests real-world knowledge of the Observable-signal interop boundary, injection-context constraints of `toSignal()`, and whether the candidate knows when `resource()` supersedes manual `toObservable` + `switchMap` plumbing.

Scenario 6: Performance issue from a poorly scoped computed()

Situation: A dashboard component has a single giant `computed()` called `dashboardViewModel` that combines ten different signals (filters, raw data list, sort order, pagination, user preferences, etc.) into one large object consumed by the template. Profiling shows the entire dashboard re-renders and recomputes heavy sorting/filtering logic every time the user toggles an unrelated "dark mode" preference signal that also happens to be read inside that same computed.

Question: Why is this an anti-pattern, and how would you restructure the computed graph to fix the performance regression?

Answer: A `computed()` re-evaluates in full whenever ANY signal it reads changes — it has no concept of "partial" recomputation. By bundling unrelated concerns (heavy filtering/sorting logic alongside a trivial dark-mode flag) into one computed, you force the expensive computation to re-run for changes that have nothing to do with it. The fix is to decompose the computed graph so each computed has the narrowest possible dependency set, following the same single-responsibility principle you'd apply to functions.

```
// ❌ WRONG – one monolithic computed, over-broad dependencies
export class DashboardBad {
  rawData = signal<Row[]>([]);
  filters = signal<Filters>({ /* ... */ });
  sortOrder = signal<'asc' | 'desc'>('asc');
  darkMode = signal(false);
  pageIndex = signal(0);

  dashboardViewModel = computed(() => {
    // Expensive: filters + sorts on EVERY dependency change, including darkMode
    let rows = this.rawData().filter(r => matchesFilters(r, this.filters()));
    rows = rows.sort((a, b) =>
      this.sortOrder() === 'asc' ? a.value - b.value : b.value - a.value
    );
    return {
      rows: rows.slice(this.pageIndex() * 20, this.pageIndex() * 20 + 20),
      darkMode: this.darkMode(), // unrelated concern baked into the same computed
    };
  });
}

// ✅ CORRECT – narrow, independent computed; each recomputes only for its own inputs
export class DashboardGood {
  rawData = signal<Row[]>([]);
  filters = signal<Filters>({ /* ... */ });
  sortOrder = signal<'asc' | 'desc'>('asc');
  darkMode = signal(false); // read directly in template/host binding, no computed needed
  pageIndex = signal(0);

  filteredRows = computed(() =>
    this.rawData().filter(r => matchesFilters(r, this.filters()))
  );

  sortedRows = computed(() =>
    [...this.filteredRows()].sort((a, b) =>
```

```

    this.sortOrder() === 'asc' ? a.value - b.value : b.value - a.value
  )
};

pagedRows = computed(() => {
  const start = this.pageIndex() * 20;
  return this.sortedRows().slice(start, start + 20);
});
}

```

This also unlocks memoization benefits: `filteredRows` only recomputes when `rawData` or `filters` change, `sortedRows` only recomputes when `filteredRows` or `sortOrder` change (toggling dark mode touches neither), and `pagedRows` only recomputes when `sortedRows` or `pageIndex` change. Angular's signal graph will skip recomputation of a `computed()` entirely if none of its tracked dependencies have changed, but that guarantee is only useful if you've actually scoped the dependencies narrowly.

Interviewer intent: Tests understanding that `computed()` memoization is dependency-set-based, not "smart" about which parts of the returned object are read downstream — architecture decomposition skill, not just API trivia.

Scenario 7: Migrating a large OnPush + RxJS component to zoneless signals incrementally

Situation: You have a 2,000-line `OrderListComponent` using `OnPush` change detection, several `BehaviorSubjects`, `combineLatest`, and manual subscriptions with `takeUntilDestroyed()`. The company is moving toward zoneless (`provideZonelessChangeDetection()`) app-wide but a full rewrite of this component in one PR is deemed too risky.

Question: Describe an incremental migration strategy for this component that keeps it shippable at every step and doesn't break under zoneless change detection mid-migration.

Answer: The safe incremental path is: (1) confirm the component doesn't rely on zone-triggered CD for things outside Angular's knowledge (e.g., raw `setTimeout` mutating DOM directly without signals/CD hooks) — those need `afterNextRender` / manual `ChangeDetectorRef.markForCheck()` calls if kept as-is; (2) convert internal Subjects to signals one state slice at a time, keeping a `toObservable()` / `toSignal()` adapter at the seams so the rest of the component is undisturbed; (3) replace `combineLatest` pipelines with `computed()` once all their sources are migrated; (4) delete the adapters once every producer/consumer on both sides is signal-based; (5) only then, verify the component is fully signal/ `input()` -driven (no un-migrated `@Input()` timing assumptions) before it needs to run correctly with zoneless CD.

```

// Step 1 (current state, unchanged):
@Component({ changeDetection: ChangeDetectionStrategy.OnPush })
export class OrderListComponent {
  private ordersSubject = new BehaviorSubject<Order[]>([]);
  private filterSubject = new BehaviorSubject<string>('');
  orders$ = combineLatest([this.ordersSubject, this.filterSubject]).pipe(
    map(([orders, filter]) => orders.filter(o => o.status.includes(filter)))
  );
}

// Step 2: introduce signals for NEW state only, bridge outward with toObservable
// so existing template bindings (async pipe) keep working unchanged.

```

```

@Component({ changeDetection: ChangeDetectionStrategy.OnPush })
export class OrderListComponentStep2 {
  private ordersSubject = new BehaviorSubject<Order[]>([]);
  filter = signal(''); // migrated slice
  private filter$ = toObservable(this.filter);

  orders$ = combineLatest([this.ordersSubject, this.filter$]).pipe(
    map(([orders, filter]) => orders.filter(o => o.status.includes(filter)))
  );
}

// Step 3: migrate the other slice too, replace combineLatest with computed,
// drop the Observable bridge entirely, template moves from `| async` to direct calls.
@Component({ changeDetection: ChangeDetectionStrategy.OnPush })
export class OrderListComponentStep3 {
  orders = signal<Order[]>([]);
  filter = signal('');

  filteredOrders = computed(() =>
    this.orders().filter(o => o.status.includes(this.filter()))
  );
}

// Step 4: once ALL components in the tree are signal-driven and no code relies
// on zone patching (setTimeout/promise auto-CD), enable zoneless at bootstrap:
bootstrapApplication(AppComponent, {
  providers: [provideZonelessChangeDetection()],
});

```

The key risk during the transition window (steps 2–3, before zoneless is enabled) is mixing Subjects and signals safely — since Zone.js is still active at that point, both models trigger CD correctly. The real danger only appears once zoneless is flipped on: anything that mutates a signal from outside an Angular-aware context (raw DOM event listener added with `addEventListener` rather than through Angular's renderer, or a non-Angular library callback) needs to explicitly call `ApplicationRef.tick()` or use `NgZone`-free-safe APIs, because there's no zone patch left to trigger CD automatically. So the migration order matters: get everything onto signals FIRST, verify all mutation entry points are Angular-aware, THEN flip zoneless.

Interviewer intent: Tests whether the candidate can plan a low-risk, PR-sized incremental migration rather than proposing a risky rewrite, and understands the specific technical precondition (all mutation entry points must be Angular-aware) that makes zoneless safe.

Scenario 8: Debugging why a `computed()` isn't updating

Situation: A `computed()` called `fullName` is supposed to update whenever `firstName` or `lastName` signals change, but the template shows a stale value even though both signals visibly change (confirmed via `console.log` inside a debug effect). The computed function itself looks correct.

Question: Walk through the most likely root causes of a `computed()` silently failing to update, in order of likelihood.

Answer:

1. **Conditional/short-circuited reads:** If the computed function reads a signal only inside a branch that isn't always taken (e.g., `if (someFlag()) { return this.firstName() + ' ' + this.lastName(); }`)

`return 'Guest';`), Angular only tracks the signals actually READ during that specific execution. If `someFlag` was false the first time, `firstName/lastName` were never registered as dependencies — until `someFlag` flips, the computed won't know to recheck them even after they change, and worse, once it does recompute for another reason, dependencies are only re-collected for that specific run's code path.

2. **Reading through `untracked()`**: If `firstName()` is accidentally wrapped in `untracked(() => this.firstName())` (common when copy-pasting effect patterns into a computed), the read no longer registers as a dependency.
3. **Object/array mutation without new reference**: If `firstName` isn't a primitive but a signal holding an object, and code does `nameState().first = 'New'` instead of `.set({ ...nameState(), first: 'New' })`, the signal's reference never changes, so `computed() / effect()` never sees a difference.
4. **Reading a stale local variable instead of calling the signal function**: e.g., destructuring `const fn = this.firstName` and later using `fn` as if it were the value instead of calling `fn()` — a TypeScript-catchable bug in strict mode, but easy to miss with `any`.
5. **The computed is never actually read anywhere reactive** — signals/computed are lazy; if nothing (template, effect, another computed) reads `fullName()`, it may not be recalculated when inspected via a non-reactive debugger step, giving the illusion it's "not updating" when in fact it's just not being pulled.

```
export class NameBadgeComponent {
  firstName = signal('Ada');
  lastName = signal('LoveLace');
  showFullName = signal(false);

  // BUG: firstName/lastName only tracked when showFullName is true
  fullName = computed(() => {
    if (!this.showFullName()) return 'Guest';
    return `${this.firstName()} ${this.lastName()}`;
  });
}
```

In this example, if `showFullName` starts `false`, `fullName` returns `'Guest'` and its only tracked dependency is `showFullName`. Changing `firstName` afterward does nothing to `fullName` (correctly, per its dependency graph) until `showFullName` flips true, at which point the computed re-runs and picks up the current values — this can look like a "stuck" computed to someone who expects a static dependency list, when really the tracked-dependency set is a function of the code path taken on the last execution. The fix here is usually to make the read unconditional so the dependency graph is stable:

```
fullName = computed(() => {
  const full = `${this.firstName()} ${this.lastName()}`;
  return this.showFullName() ? full : 'Guest';
});
```

Interviewer intent: This is a deep-diagnostics question testing whether the candidate understands that Angular's signal dependency tracking is dynamic and per-execution (not a static declared list), which is the single most common cause of "my computed isn't updating" bugs reported in the wild.

Scenario 9: Using `linkedSignal` for resettable derived state

Situation: A multi-step wizard has a `selectedShippingMethod` signal that should default to the cheapest available option whenever the `availableMethods` list changes (e.g., after the user changes their delivery address), but should still let the user manually override it afterward within the same address selection.

Question: Why is a plain `computed()` insufficient here, and how does `linkedSignal()` solve it?

Answer: A plain `computed()` can't work here because it's read-only and always derives strictly from its sources — you can never let the user "override" it, since any manual `.set()` isn't possible on a `computed`. A plain writable `signal()` combined with an `effect()` that resets it when `availableMethods` changes is the naive approach, but that reintroduces the effect-writing-state anti-pattern (timing lag, and needing to guard against re-triggering). `linkedSignal()` is purpose-built for exactly this "writable signal that resets to a computed default when its source changes, but remains manually overridable in between" pattern.

```
export class ShippingStepComponent {
  availableMethods = signal<ShippingMethod[]>([]);

  // Resets to the cheapest method automatically whenever availableMethods changes,
  // but can still be overridden with .set() by the user in between changes.
  selectedShippingMethod = linkedSignal(() => {
    const methods = this.availableMethods();
    return methods.length > 0
      ? methods.reduce((a, b) => (a.cost < b.cost ? a : b))
      : null;
  });

  onUserPicksMethod(method: ShippingMethod): void {
    this.selectedShippingMethod.set(method); // manual override, persists
  }

  onAddressChanged(newMethods: ShippingMethod[]): void {
    this.availableMethods.set(newMethods); // linkedSignal recomputes default, override
    // cleared
  }
}
```

The distinction that matters in interviews: `computed()` is pure-derived and never writable; `linkedSignal()` is writable but has a "source of truth reset" baked in, replacing what used to require an `effect()` + plain `signal()` combo (which was both more code and prone to the write-during-effect timing issues from Scenario 2/3).

Interviewer intent: Tests awareness of newer signal primitives beyond the core three, and whether the candidate reaches for the right tool instead of reimplementing `linkedSignal`'s behavior manually with `effect()` (a common "doesn't know the new API" tell).

Scenario 10: input() vs @Input() and reacting to input changes

Situation: A component previously used `@Input() userId!: string;` combined with `ngOnChanges` to refetch user data whenever `userId` changed. You're asked to migrate it to the signal-based `input()` API and preserve the same refetch-on-change behavior.

Question: How do you replace the `ngOnChanges` refetch logic using signal inputs, and what's the idiomatic way to trigger a refetch reactively?

Answer: `input()` produces a read-only signal, so instead of an imperative `ngOnChanges` lifecycle hook, you derive/react to it declaratively — either with a `computed()` / `resource()` if the "refetch" is itself an async derivation of the input (best), or, only if a true side effect outside the signal graph is needed, with `effect()`.

```
// Before: @Input + ngOnChanges
@Component({ /* ... */ })
export class UserProfileOld implements OnChanges {
  @Input() userId!: string;
  user?: User;

  ngOnChanges(changes: SimpleChanges): void {
    if (changes['userId']) {
      this.userService.getUser(this.userId).subscribe(u => this.user = u);
    }
  }
}

// After: input() + resource(), no lifecycle hook, no manual subscription management
@Component({ /* ... */ })
export class UserProfileNew {
  userId = input.required<string>();
  private userService = inject(UserService);

  userResource = resource({
    params: () => ({ id: this.userId() }),
    loader: ({ params, abortSignal }) =>
      this.userService.getUserAsync(params.id, abortSignal),
  });

  user = computed(() => this.userResource.value());
}
```

If the "refetch" were instead a genuine imperative side effect unrelated to producing a value (e.g., pushing an analytics event whenever `userId` changes), `effect(() => { const id = this.userId(); untracked(() => this.analytics.trackView(id)); })` would be appropriate — but for anything that produces data the template displays, prefer `computed()` / `resource()` over `effect()`, per the same principle from Scenario 3.

Interviewer intent: Confirms the candidate knows `input()` replaces both `@Input()` and the `ngOnChanges` reactivity model, and reinforces the recurring theme of choosing `computed/resource` over `effect` for value derivation.

Scenario 11: model() for two-way binding replacing @Input/@Output pair

Situation: A custom `<app-quantity-picker>` component currently exposes `@Input() value: number` and `@Output() valueChange = new EventEmitter<number>()` to support `[(value)]` banana-in-a-box binding from parent templates. You're asked to modernize it with signals while preserving the two-way binding contract.

Question: How does `model()` simplify this, and what should you watch out for regarding writes from both the parent and the child?

Answer: `model()` creates a signal that is both readable and writable from within the component, and automatically wires up the paired `output (valueChange)` needed for `[(value)]` syntax — collapsing the `@Input / @Output` pair into a single declaration.

```
@Component({
  selector: 'app-quantity-picker',
  template: `
    <button (click)="decrement()">-</button>
    <span>{{ value() }}</span>
    <button (click)="increment()">+</button>
  `,
})
export class QuantityPickerComponent {
  value = model<number>(1); // readable + writable, auto-emits valueChange

  increment(): void {
    this.value.update(v => v + 1); // parent's bound value updates too
  }

  decrement(): void {
    this.value.update(v => Math.max(0, v - 1));
  }
}
```

```
<!-- Parent template – identical binding syntax to the old @Input/@Output pair -->
<app-quantity-picker [(value)]="cartQuantity" />
```

The gotcha: because `model()` is writable from BOTH sides (parent can `.set()` it directly if it exposes the signal, and the child updates it internally), you must be disciplined about which side "owns" validation. If the parent binds a signal with its own constraints (e.g., max stock), the child's `increment()` blindly calling `update(v => v + 1)` can push it past a business rule the parent enforces elsewhere — with two-way `@Input() / @Output()` this risk existed too, but it's easy to forget it didn't go away just because the API got shorter. The fix is usually to keep validation logic inside the child (as shown, clamping at 0) or to expose the model as `model.required<number>()` with a documented contract for valid ranges, and let the parent read `computed()` values derived from it rather than mutating it externally in ways the child doesn't expect.

Interviewer intent: Tests knowledge of the newer `model()` API and whether the candidate recognizes that two-way binding's classic "who owns the source of truth" problem persists regardless of API sugar.

Scenario 12: Signal equality function and NaN/object comparison pitfalls

Situation: A `signal<Point>({ x: 0, y: 0 })` is updated every animation frame via `.set({ x: newX, y: newY })`, but a downstream `computed()` and template binding barely ever re-render, even though the numbers are visibly changing according to a separate debug log outside Angular's reactivity.

Question: What's the most likely cause, and how do you fix a signal holding object values that should trigger updates on every distinct value?

Answer: By default, `signal()` uses `Object.is` equality to decide whether a `.set()` call actually changed the value and should notify consumers. If code mistakenly reuses/mutates the same object reference before calling `.set()` (e.g., `point.x = newX; pointSignal.set(point);` where `point` is the same object retrieved via `pointSignal()`), `Object.is` sees the identical reference and treats it as unchanged — wait, actually since it's the same reference, `Object.is` returns true (equal), so the signal correctly does NOT notify, which the developer perceives as "not updating" but is actually working exactly as documented: same reference in means "no change" was detected.

The fix is to always pass a new object into `.set()` / `.update()`:

```
export class PointTracker {
  point = signal<Point>({ x: 0, y: 0 });

  // ❌ WRONG - mutates the object obtained from the signal, same reference passed back
  moveWrong(dx: number, dy: number): void {
    const p = this.point();
    p.x += dx;
    p.y += dy;
    this.point.set(p); // Object.is(p, p) === true → no update fires
  }

  // ✅ CORRECT - new object reference every time
  moveRight(dx: number, dy: number): void {
    this.point.update(p => ({ x: p.x + dx, y: p.y + dy }));
  }
}
```

A related but distinct pitfall: if you legitimately need custom equality (e.g., comparing by value for a large immutable record, or intentionally treating `NaN === NaN` as equal, which `Object.is` already does correctly unlike `===`), `signal()` accepts an `equal` option:

```
const point = signal<Point>({ x: 0, y: 0 }, {
  equal: (a, b) => a.x === b.x && a.y === b.y, // custom deep-ish comparison
});
```

Be cautious with custom `equal` functions that are expensive (e.g., deep-equal on large arrays) — they run on every `.set()` call and can become their own performance bottleneck if overused on hot-path signals like animation frames.

Interviewer intent: Tests precise understanding of signal change-detection semantics (`Object.is` by default) versus the common misconception that signals do deep equality, and awareness of the configurable `equal` option.

Scenario 13: afterRenderEffect / DOM measurement with signals

Situation: A component needs to measure the rendered width of a dynamically-sized `<div>` (dependent on a signal-driven text content) and store it in a signal for use in a layout calculation elsewhere, without causing `ExpressionChangedAfterItHasBeenCheckedError`-style issues or excessive reflows.

Question: What's the correct signals-idiomatic way to read layout/DOM state after render and feed it back into the signal graph?

Answer: Use `afterRenderEffect()` (or `afterNextRender / afterRender` for one-off or continuous read/write phases), which is specifically designed to run after Angular has finished rendering, avoiding the reentrant-CD issues of trying to read DOM measurements inside a regular `effect()` (which can run before layout is stable, and repeatedly triggering writes from it risks the same infinite-loop class of bug as Scenario 2, just now involving the DOM).

```
@Component({
  selector: 'app-measured-box',
  template: `<div #box>{{ label() }}</div>`,
})
export class MeasuredBoxComponent {
  label = signal('Hello world');
  measuredWidth = signal(0);

  private boxRef = viewChild.required<ElementRef<HTMLDivElement>>('box');

  constructor() {
    afterRenderEffect(() => {
      // Runs after Angular has committed DOM changes for this render pass –
      // safe to read layout without triggering ExpressionChangedAfterItHasBeenCheckedError.
      const width = this.boxRef().nativeElement.getBoundingClientRect().width;
      // Guard against redundant writes causing another render pass unnecessarily:
      if (width !== this.measuredWidth()) {
        this.measuredWidth.set(width);
      }
    });
  }
}
```

Why not a plain `effect()`? Because a plain `effect()` scheduling timing relative to render/layout isn't guaranteed the same way — `afterRenderEffect` explicitly hooks into Angular's render lifecycle phases (`earlyRead`, `write`, `mixedReadWrite`, `read` under the hood), which is the correct primitive for read-DOM-after-render use cases, whereas generic `effect()` is meant for signal-graph side effects, not DOM-timing-sensitive work.

Interviewer intent: Tests whether the candidate knows about render-lifecycle-aware effect variants beyond plain `effect()`, relevant for anyone doing custom layout/measurement work in a modern Angular + signals app.

Scenario 14: Signal-based state machine for a multi-step wizard (avoiding effect for transitions)

Situation: A checkout wizard has steps `'cart' | 'shipping' | 'payment' | 'confirm'`. The original implementation used an `effect()` that watched a `currentStepIndex` signal and imperatively called methods like `resetPaymentForm()` or `validateShippingBeforeLeaving()` depending on which transition occurred, leading to bugs where transitions fired twice or skipped validation depending on batching timing.

Question: How would you redesign this as an explicit signal-driven state machine that avoids relying on `effect()` to orchestrate business-logic transitions?

Answer: Transition logic (validation, side effects tied to a specific transition, not just "step changed") belongs in an explicit transition function/action, not in an `effect()` reacting after the fact to a state change. Effects are good for genuinely passive reactions (e.g., logging analytics whenever the step changes) but bad for anything where order, exactly-once execution, or "should this transition even be allowed" logic matters — because effects run after the write has already committed, so you can't "veto" a transition from inside an effect, and batching can coalesce multiple rapid writes into a single effect run, skipping intermediate states the business logic assumed it would see.

```
type Step = 'cart' | 'shipping' | 'payment' | 'confirm';

@Injectable({ providedIn: 'root' })
export class CheckoutWizardService {
  private readonly _step = signal<Step>('cart');
  readonly step = this._step.asReadonly();

  private readonly stepOrder: Step[] = ['cart', 'shipping', 'payment', 'confirm'];

  shippingForm = signal<ShippingInfo | null>(null);
  paymentForm = signal<PaymentInfo | null>(null);

  readonly canGoNext = computed(() => {
    switch (this._step()) {
      case 'cart': return true;
      case 'shipping': return this.shippingForm() !== null;
      case 'payment': return this.paymentForm() !== null;
      case 'confirm': return false;
    }
  });

  // Explicit, synchronous, exactly-once transition method – NOT an effect.
  goNext(): boolean {
    if (!this.canGoNext()) return false;

    const currentIndex = this.stepOrder.indexOf(this._step());
    const next = this.stepOrder[currentIndex + 1];
    if (!next) return false;

    // Side effects tied to a SPECIFIC transition, run exactly once, in order,
    // and can veto/rollback before committing the step change.
    if (this._step() === 'shipping' && next === 'payment') {
      this.resetPaymentForm();
    }
  }
}
```

```

    }

    this._step.set(next);
    return true;
  }

  private resetPaymentForm(): void {
    this.paymentForm.set(null);
  }
}

// A passive, non-business-critical reaction (analytics) IS an appropriate effect() use:
@Component({ /* ... */ })
export class CheckoutWizardComponent {
  wizard = inject(CheckoutWizardService);
  private analytics = inject(AnalyticsService);

  constructor() {
    effect(() => {
      const step = this.wizard.step();
      untracked(() => this.analytics.trackPageview(`checkout/${step}`));
    });
  }
}

```

Interviewer intent: Tests the ability to distinguish "reactive side effect" (fine for effect()) from "business transition logic requiring exactly-once ordering and vetoability" (belongs in an explicit method), a distinction many candidates blur when over-applying signals/effects to everything.

Scenario 15: Untracked reads to avoid unwanted dependencies in effect()

Situation: An `effect()` is meant to save form data to `localStorage` only when the form's `data` signal changes, but it should also include a `lastSavedBy` signal (current user) in the payload without causing the effect to re-run merely because the user identity changes mid-session (which happens rarely but shouldn't itself trigger a save).

Question: How do you read `lastSavedBy` inside the effect without making it a reactive dependency, and why does this matter?

Answer: Wrap the read of `lastSavedBy` in `untracked()`. Angular's effect dependency tracking is based on which signals are read synchronously (and NOT inside `untracked()`) during the function body's execution — `untracked()` lets you read a signal's current value for use in logic without registering it as something that should re-trigger the effect on its own.

```

@Component({ /* ... */ })
export class FormAutosaveComponent {
  formData = signal<FormData | null>(null);
  currentUser = signal<User | null>(null);

  constructor() {
    effect(() => {
      const data = this.formData(); // tracked - effect re-runs when this changes
    });
  }
}

```

```

    if (!data) return;

    // untracked: we want the CURRENT user at save-time, but a user change
    // alone should not itself trigger a save.
    const user = untracked(() => this.currentUser());

    localStorage.setItem('draft', JSON.stringify({ data, savedBy: user?.id }));
  });
}
}

```

Without `untracked()`, every time `currentUser` changes (e.g., a session refresh event updates cached user metadata unrelated to form editing), the effect would spuriously re-save the form with whatever `formData` happens to be at that moment — not wrong data, but semantically incorrect ("saved because user changed" when the intent was "saved because the form changed"), and in a system with save-rate telemetry or optimistic-locking, this can cause confusing duplicate save events. The general rule: track only the signals whose CHANGE should be a trigger; read-but-don't-trigger-on values belong in `untracked()`.

Interviewer intent: Tests fine-grained understanding of dependency tracking scoping within `effect()` bodies and the practical reason `untracked()` exists, beyond "it's an API that exists."

Scenario 16: Signal store pattern (feature-level signals) vs NgRx for a mid-size app

Situation: A team currently uses NgRx for a mid-size app (roughly 15 feature modules) and finds the actions/reducers/selectors boilerplate slowing down velocity for simple CRUD features, while a few complex features (undo/redo, time-travel debugging needs) genuinely benefit from NgRx's tooling. Leadership asks for a recommendation on introducing a signal-based store pattern.

Question: How would you decide which features migrate to a lightweight signal-based store versus staying on NgRx, and what does a signal store look like for a simple feature?

Answer: The decision axis isn't "signals are newer, use them everywhere" — it's about which architectural guarantees each feature actually needs: (1) features needing time-travel debugging, action replay, complex undo/redo, or strict action-log auditability (e.g., regulatory audit trail features) benefit from NgRx's action-based architecture and DevTools integration; (2) simple CRUD/local-UI-state features (filters, toggles, list views, form drafts) are usually significantly simpler as a signal-based store with plain classes exposing readonly signals and computed selectors, without the ceremony of actions/reducers/effects/selectors for logic that's fundamentally "call a method, update a signal."

```

// A lightweight signal store for a simple feature – no actions, no reducers,
// just a class exposing readonly signals + computed selectors + methods.
@Injectable({ providedIn: 'root' })
export class TaskListStore {
  private readonly _tasks = signal<Task[]>([]);
  private readonly _filter = signal<'all' | 'active' | 'done'>('all');
  private readonly _loading = signal(false);

  readonly tasks = this._tasks.asReadonly();
  readonly filter = this._filter.asReadonly();
  readonly loading = this._loading.asReadonly();

  readonly visibleTasks = computed(() => {

```

```

    const f = this._filter();
    const all = this._tasks();
    if (f === 'active') return all.filter(t => !t.done);
    if (f === 'done') return all.filter(t => t.done);
    return all;
  });

  readonly stats = computed(() => {
    const all = this._tasks();
    return { total: all.length, done: all.filter(t => t.done).length };
  });

  private taskService = inject(TaskService);

  async load(): Promise<void> {
    this._loading.set(true);
    try {
      this._tasks.set(await this.taskService.fetchAll());
    } finally {
      this._loading.set(false);
    }
  }

  toggleDone(id: string): void {
    this._tasks.update(tasks =>
      tasks.map(t => (t.id === id ? { ...t, done: !t.done } : t))
    );
  }

  setFilter(filter: 'all' | 'active' | 'done'): void {
    this._filter.set(filter);
  }
}

```

The migration recommendation: keep NgRx for the complex/audited features (avoid a risky rewrite of working, well-tested reducer logic just for aesthetics), migrate simple CRUD features to signal stores incrementally as they're touched for other reasons, and standardize on the pattern above as the team's "lightweight store" convention so the codebase doesn't end up with N different homemade signal-store shapes.

Interviewer intent: Tests architectural judgment — whether the candidate can evaluate signals vs. a full state-management library on actual requirements (time-travel, audit, complexity) rather than defaulting to "signals are simpler, remove NgRx everywhere," which is itself a red flag answer.

Scenario 17: Testing components/services with signals (avoiding fakeAsync overuse)

Situation: A QA lead complains that unit tests for a signal-based `computed()` chain are flaky — sometimes assertions run before the computed has "caught up," and the previous team's habit of wrapping every test in `fakeAsync` + `tick()` (carried over from RxJS testing habits) doesn't reliably fix it, and sometimes it's cargo-culted in unnecessarily.

Question: How should signals actually be tested, and when (if ever) is `fakeAsync/tick` genuinely needed in a signals test?

Answer: `computed()` values are synchronous and pull-based — reading `mySignal()` after a `.set()` call always reflects the latest state immediately, with zero need for `fakeAsync` / `tick()` / `flush()`. Flakiness in "pure signal" tests almost always indicates a test that's actually exercising something asynchronous elsewhere (an HTTP call behind a `resource()`, a `toSignal()`-wrapped Observable, or a genuine `effect()` whose scheduling is intentionally deferred to a microtask) rather than the computed itself.

```
describe('CartService (pure signal logic – synchronous, no fakeAsync needed)', () => {
  it('updates total when item is added', () => {
    const service = TestBed.inject(CartService);
    service.addItem({ id: '1', qty: 2, price: 10 });
    expect(service.total()).toBe(20); // synchronous read, no tick() needed
  });
});

describe('CheckoutEffectComponent (genuine effect() – needs flush via TestBed)', () => {
  it('logs analytics when step changes', () => {
    const fixture = TestBed.createComponent(CheckoutWizardComponent);
    fixture.detectChanges(); // triggers initial effect run

    const analyticsSpy = spyOn(TestBed.inject(AnalyticsService), 'trackPageView');
    fixture.componentInstance.wizard.goNext();

    // effect() runs on a scheduled microtask flush, not immediately after set() –
    // TestBed.flushEffects() (or awaiting a microtask / calling fixture.detectChanges()
    // again in newer Angular test harnesses) is the correct tool, NOT fakeAsync/tick.
    TestBed.flushEffects();

    expect(analyticsSpy).toHaveBeenCalledWith('checkout/shipping');
  });
});

describe('UserProfileNew (resource() – genuinely async, needs awaiting)', () => {
  it('loads user data', async () => {
    const fixture = TestBed.createComponent(UserProfileNew);
    fixture.componentRef.setInput('userId', 'u1');
    fixture.detectChanges();

    await fixture.whenStable(); // resource's underlying fetch is real async work

    expect(fixture.componentInstance.user()).toEqual({ id: 'u1', name: 'Ada' });
  });
});
```

The rule of thumb for interviews and for real codebases: `fakeAsync` / `tick()` exists to control the JS macrotask/microtask queue and timers for RxJS/Promise-based code — it has nothing to do with `computed()`, which needs no queue draining at all. Reach for `TestBed.flushEffects()` (or the equivalent stabilization mechanism) specifically for `effect()`, and `await fixture.whenStable()` / real `await` for genuinely async work like `resource()` or Promise-returning methods. Cargo-culting `fakeAsync` everywhere "because it fixed flakiness once" usually just masks a different bug (e.g., an effect not yet flushed) rather than actually needing time control.

Interviewer intent: Tests practical unit-testing competence with the signals APIs specifically — many

candidates know signals conceptually but haven't actually had to test `effect()/resource()` timing correctly, which is exactly where teams get bitten in CI.

Scenario 18: Deep-diving why a shared `computed()` across two components behaves unexpectedly

Situation: Two sibling components both inject the same `InventoryService` and both read `service.lowStockItems` (a `computed()`). One component also calls `service.refreshPrices()` internally, which internally does a `.set()` on a `prices` signal that `lowStockItems` doesn't even depend on. Despite that, the other, unrelated component sees a re-render whenever `refreshPrices()` is called, and the team can't figure out why since `lowStockItems`'s `computed` body clearly doesn't reference `prices`.

Question: What's the most likely explanation, and how would you diagnose and fix it?

Answer: The most likely explanation is that the `computed`'s dependency set isn't actually as narrow as the source code visually suggests — often because `lowStockItems` calls a shared helper function or reads a signal indirectly (e.g., through another `computed`, or via a getter on a plain object that itself happens to read `prices()` under the hood, such as an `item.isLowStock` getter that factors in price-based reordering thresholds). Since dependency tracking is based on ANY signal read during the `computed`'s actual execution — including reads that happen inside functions called by the `computed`, not just top-level reads in the lambda — a seemingly-unrelated helper can silently widen the dependency set.

```
@Injectable({ providedIn: 'root' })
export class InventoryService {
  items = signal<Item[]>([]);
  prices = signal<Record<string, number>>({});

  // Looks independent of `prices` at a glance...
  lowStockItems = computed(() =>
    this.items().filter(item => this.isLowStock(item))
  );

  // ...but the helper it calls DOES read `prices()` – widening the dependency set!
  private isLowStock(item: Item): boolean {
    const reorderThreshold = this.prices()[item.id] > 100 ? 5 : 10; // hidden read
    return item.quantity < reorderThreshold;
  }

  refreshPrices(): void {
    this.prices.set(fetchLatestPrices());
  }
}
```

Diagnosis approach: use Angular DevTools' signal graph inspector (or, absent that, temporarily log inside the `computed` with a counter to confirm re-execution count vs. expectation), and audit every function call made from within the `computed` body for hidden signal reads — this is the signals equivalent of an RxJS `combineLatest` silently including an extra stream you forgot was in the pipeline. The fix, once identified, is either to make the dependency intentional and documented (if it's actually correct business logic that low-stock status DOES depend on price, in which case the "unexpected" re-render was actually correct and the surprise was a documentation/naming gap) or to refactor `isLowStock` to not depend on `prices` if that was unintended, restoring the narrower dependency set.

Interviewer intent: Tests advanced debugging skill: understanding that `computed()` dependency tracking is transitive through any function calls made during its execution, not just literal signal reads visible in the `computed`'s own lambda — a nuance that trips up even experienced developers.

Scenario 19: Signal-based debounced search without RxJS operators

Situation: A search box needs to debounce keystrokes by 300ms before firing an API call, something the team previously did with `Subject` + `debounceTime` + `switchMap`. The team wants to know if this is reasonably done with pure signals or if RxJS is still the right tool here.

Question: Can this be done with signals alone, and what's the honest tradeoff versus keeping RxJS for this specific use case?

Answer: It CAN be done with signals plus a manual timer, but debouncing is fundamentally a time-based stream operation, and RxJS's `debounceTime` is a mature, well-tested, cancellation-safe primitive for exactly this — reimplementing it by hand with `setTimeout` inside an `effect()` is more code, more edge cases to get right (clearing the previous timeout, handling component destruction, avoiding leaks), for no real benefit over the well-known RxJS pattern. The honest, senior-level answer is: **use `toObservable()` to bridge into RxJS for the debounce operator specifically, then bridge back with `toSignal()` (or `resource()`) — don't reimplement debounce from scratch just to stay "pure signals."**

```
@Component({ /* ... */ })
export class SearchBoxComponent {
  query = signal('');

  private debouncedQuery$ = toObservable(this.query).pipe(
    debounceTime(300),
    distinctUntilChanged(),
  );

  private debouncedQuery = toSignal(this.debouncedQuery$, { initialValue: '' });

  searchResults = rxResource({
    params: () => ({ q: this.debouncedQuery() }),
    stream: ({ params }) =>
      params.q.length > 0 ? this.searchService.search$(params.q) : of([]),
  });
}

// A hand-rolled signals-only version – technically possible, but reinvents
// debounce with more surface area for bugs (cleanup, cancellation on destroy):
@Component({ /* ... */ })
export class SearchBoxManualDebounce {
  query = signal('');
  private debouncedQuery = signal('');
  private timeoutId?: ReturnType<typeof setTimeout>;
  private destroyRef = inject(DestroyRef);

  constructor() {
    effect(() => {
      const q = this.query(); // tracked
      clearTimeout(this.timeoutId);
```

```

    this.timeoutId = setTimeout(() => this.debounceQuery.set(q), 300);
  });

  this.destroyRef.onDestroy(() => clearTimeout(this.timeoutId));
}
}

```

The RxJS-bridged version is shorter, leverages `distinctUntilChanged` and `debounceTime` implementations that are battle-tested for edge cases (rapid destroy/recreate, zero-delay debounce, etc.), and composes naturally with `rxResource()`'s built-in cancellation of stale searches. The hand-rolled version works but is "reinventing a wheel that RxJS already ships," which is a meaningful maintenance cost for zero benefit — the correct engineering judgment is knowing when to reach outside the signals API rather than treating "no RxJS" as a dogmatic goal.

Interviewer intent: Tests whether the candidate treats signals as a religion ("must avoid RxJS entirely") versus a pragmatic engineer who bridges to RxJS specifically for the operators (time-based, complex flow control) that signals don't natively provide.

Scenario 20: Zoneless app crashing because a third-party library mutates a signal-backed value outside Angular's knowledge

Situation: After enabling `provideZonelessChangeDetection()`, a component using a third-party charting library (which internally uses its own event loop / `requestAnimationFrame` callbacks to update a shared data object) stops reflecting live updates in the Angular-rendered legend, even though the chart itself animates fine (since it manages its own canvas rendering outside Angular).

Question: Why does zoneless change detection break this specific integration, and how do you fix it correctly?

Answer: Under zone-based change detection, ANY async callback (including third-party `requestAnimationFrame` / timer callbacks) would trigger Angular's zone-patched APIs and eventually cause a CD sweep, which "accidentally" kept the Angular-rendered legend in sync even though nothing in Angular's own reactivity graph was touched. Under zoneless, there is no such patching — Angular only re-renders when a signal it's tracking changes (or a manual `ApplicationRef.tick()` is called), so if the third-party library mutates its own internal data and the Angular template reads that data through a plain property (not a signal) or reads a signal that's never actually `.set()` when the library's callback fires, nothing tells Angular a render is needed.

The fix is to make the boundary between the third-party library's callback and Angular's reactivity explicit: have the library's update callback explicitly write into a signal.

```

@Component({
  selector: 'app-live-chart',
  template: `<div class="legend">Latest: {{ latestValue() }}</div>`,
})
export class LiveChartComponent implements OnInit, OnDestroy {
  latestValue = signal(0);
  private chart?: ThirdPartyChart;

  ngOnInit(): void {
    this.chart = new ThirdPartyChart(this.canvasEl, {
      // The library's own RAF-driven callback – Angular has NO visibility into this.

```

```

    onDataUpdate: (value: number) => {
      // Explicitly bridge into the signal graph so Angular knows to re-render.
      this.latestValue.set(value);
    },
  });
}

ngOnDestroy(): void {
  this.chart?.destroy();
}
}

```

If the library's callback fires very frequently (e.g., 60fps) and writing a signal on every frame is more churn than the legend actually needs to display, throttle the signal write itself (e.g., only `.set()` every N frames or when the value crosses a display-relevant threshold) rather than relying on zone-based CD's implicit (and wasteful) "re-check everything on every callback" behavior — this is actually one of the real performance wins of going zoneless: you're forced to be explicit about what should trigger a render, instead of silently re-rendering the whole tree on every unrelated async callback the way zone.js did.

Interviewer intent: Tests whether the candidate understands the fundamental contract change zoneless introduces (rendering is now driven exclusively by explicit signal writes or manual `tick()`, not implicit "anything async happened" triggers), and can identify/fix the resulting integration gaps with non-Angular-aware third-party code — a very real production migration pitfall.

Quick Revision Cheat Sheet

- `computed()` **for values**, `effect()` **for side effects**. If a piece of state can be expressed as a pure function of other signals, it must be a `computed()`. Never use `effect()` to "derive" a value into a plain signal — it introduces scheduling lag and risks infinite loops if it writes back to something it reads.
- **Writing to a signal you're reading inside the same `effect()` / `computed()` causes infinite loops or NG0600**. Break the cycle by deriving with `computed()` instead, or restructure so the write target is never also a tracked read.
- **Signals use `Object.is` equality by default**. Mutating an object/array in place and calling `.set()` with the same reference will NOT notify consumers — always pass a new reference (spread/ `map` / `filter`), or supply a custom `equal` function if value-based comparison is genuinely needed.
- `computed()` **dependency tracking is dynamic and per-execution, not a static declared list**. Conditional reads mean a dependency is only tracked when that code path actually runs; and dependencies picked up through helper function calls inside the computed still count — the "hidden dependency" bug class.
- `resource()` / `rxResource()` **are the correct tools for async-derived state** (HTTP calls, cancellation-sensitive fetches) — they handle request cancellation, stale-response discarding, and loading/error state natively, which a hand-rolled `effect()` + `Promise` / `fetch` combo reimplements poorly and race-condition-prone.
- `toSignal()` / `toObservable()` **are the interop boundary with RxJS** — necessary for legacy Observable-based services and for operators signals don't natively have (`debounceTime`, `combineLatest` -style flow control). Bridging to RxJS for these specific needs is pragmatic engineering, not a failure to "go pure signals."
- `linkedSignal()` solves the "writable signal with a resettable computed default" pattern (e.g., auto-

select cheapest option but allow manual override) without the effect-writes-state anti-pattern.

- `input() / model() / output()` replace `@Input() / @Output() / ngOnChanges` reactivity — `model()` specifically replaces the `@Input + @Output` two-way-binding pair, but the "who owns validation" problem from two-way binding doesn't disappear just because the API is shorter.
- `untracked()` lets you read a signal's current value inside an `effect() / computed()` without registering it as a dependency — use it for "read but don't trigger" values (e.g., current user ID at save time, unrelated to what triggers the save).
- **Scope `computed()` dependency sets narrowly.** A single giant computed combining unrelated concerns (heavy filtering + a trivial UI flag) recomputes the expensive part whenever ANY input changes — decompose into a chain of narrow computeds for correct memoization.
- **Business-transition logic (validation, exactly-once side effects, vetoable transitions) belongs in explicit methods, not `effect()`.** Effects run after a write commits and can be coalesced by batching — they can't veto a change and aren't guaranteed to see every intermediate state.
- **Testing signals is synchronous** — no `fakeAsync / tick()` needed for pure `computed() / signal()` logic. Use `TestBed.flushEffects()` for `effect()` timing and real `await / whenStable()` for genuinely async `resource()`-backed state. Cargo-culting `fakeAsync` everywhere usually masks a different underlying bug.
- **Zoneless change detection removes the implicit "anything async re-renders everything" safety net.** Every mutation entry point — especially third-party library callbacks (rAF loops, non-Angular event listeners) — must explicitly write to a signal (or call `ApplicationRef.tick()`) for Angular to know a render is needed. Migrate incrementally: convert state to signals first, verify all mutation entry points are Angular-aware, and only then enable `provideZonelessChangeDetection()`.
- `afterRenderEffect() / afterNextRender()`, not plain `effect()`, are the correct primitives for DOM-measurement/layout-timing work that must run after Angular commits a render pass.

Created By - Durgesh Singh